

1. Problem Description

1.1 Context & Motivation

Growing up, my mother was a nutritionist, so I have been raised with a strict awareness of dietary balance. However, maintaining these standards as a university student has proven a significant struggle due to the immense workload and limited finances. This challenge is uniquely exacerbated by the Minerva rotation model where every four months, we migrate to a new city with a vastly different economic landscape.

1.2 The Optimization Challenge

The mental effort required to manually balance food costs against macronutrient targets, especially in a new currency, is highly inefficient. Moreover, while the classical *Stigler Diet* problem provides a valuable theoretical foundation in optimization, its standard formulation relies on assumptions that are too idealized for realistic dietary planning. It utilizes Linear Programming (LP), which assumes continuous variables (allowing unrealistic fractional purchases like 0.37 jars of peanut butter) and ignores physical logistics.

1.3 Proposed Solution

I propose the "Minervan Diet Optimizer (MDO)," a Generalized Mixed-Integer Linear Programming (MILP) framework. Using local price data as input, the tool produces a cost-optimal and logically consistent weekly grocery basket that can be readily adapted across locations, allowing a Minervan to maintain an optimal diet regardless of their rotation city.

2. Solution Specification

2.1 Mathematical Formulation

We define the set of available food items as $J = \{1, \dots, N\}$. For each item j , we define the cost c_j , protein content p_j , energy content e_j , and a set of category tags C_j (e.g., Vegetables, Dairy).

Decision Variables: To model the discrete nature of grocery shopping, we employ two sets of variables:

- $x_j \in \mathbb{Z}_{\geq 0}$: The integer quantity of item j purchased.
- $y_j \in \{0, 1\}$: A binary indicator variable, where $y_j = 1$ if item j is purchased ($x_j \geq 1$), and 0 otherwise.

Objective Function: The objective is to minimize the total cost Z of the weekly grocery basket, defined as the sum of the unit price c_j of each food item multiplied by the integer quantity purchased x_j :

$$\min_{x,y} Z = \sum_{j=1}^N c_j x_j.$$

This formulation reflects the practical goal of cost minimization in local currency, where costs accrue only through purchased quantities. The binary indicator variable y_j does not appear directly in the objective, as it serves exclusively to enforce logical relationships and category-level requirements within the constraint set rather than contributing to cost.

2.2 Constraints & Justification

A. Nutrition (WHO Standards): To ensure the model is generalizable to a diverse, anonymous student body, I utilize World Health Organization (WHO) standards.

Protein (P_{req}): Based on the WHO standard of 0.83g/kg, daily requirements are approx. 62g (Male, 75kg) and 46g (Female, 55kg). I target the average: 54g/day (378g/week).

Energy (E_{req}): Using UK government reference points of 2,500 kcal (M) and 2,000 kcal (F), I target the average: 2,250 kcal/day (15,750 kcal/week).

$$\sum_{j=1}^N p_j x_j \geq 378 \quad , \quad \sum_{j=1}^N e_j x_j \geq 15750$$

B. The "Backpack" Constraint (Logistics): Unlike standard household optimization models, Minerva students must transport groceries manually without a vehicle. To prevent the solver from selecting 50 low-density items (e.g., 50 bags of spinach), I enforce a capacity constraint $K_{max} = 15$ units, representing the physical limit of a student backpack/tote bag:

$$\sum_{j=1}^N x_j \leq 15$$

C. Category Diversity (Cardinality): To ensure micronutrient balance, I enforce variety thresholds (V_k) for key categories. This forces the model to construct a balanced grocery basket rather than a "monodiet" of the cheapest calorie source.

$$\sum_{j \in \text{Vegetables}} y_j \geq 2 \quad , \quad \sum_{j \in \text{Fruits}} y_j \geq 1 \quad , \quad \sum_{j \in \text{Protein}} y_j \geq 2 \quad , \quad \sum_{j \in \text{Dairy}} y_j \geq 1$$

D. Linking Constraints (Variable Activation & Minimum Purchase): To mathematically link the integer quantity x_j to the binary indicator y_j , we utilize a dual-bounding formulation. This ensures that the decision to select an item ($y_j = 1$) strictly corresponds to purchasing a non-zero quantity within a feasible range $[1, M]$.

$$y_j \leq x_j \leq M \cdot y_j \quad \forall j \in J$$

1. Upper Bound (Big-M):

$$x_j \leq M \cdot y_j \quad \forall j \in J$$

We set $M = 50$, representing the maximum physical capacity for any single item.

- **Inactive Case:** If $y_j = 0$, then $x_j \leq 0$, forcing $x_j = 0$.
- **Active Case:** If $y_j = 1$, then $x_j \leq 50$, leaving the quantity effectively unconstrained within the backpack limit.

2. Lower Bound (Minimum Buy):

$$x_j \geq y_j \quad \forall j \in J$$

This constraint prevents the "Ghost Basket" anomaly, where the model might set $y_j = 1$ to satisfy diversity constraints (e.g., "Select 2 Vegetables") but set $x_j = 0$ to minimize cost.

- **Active Case:** If $y_j = 1$, then $x_j \geq 1$, forcing the purchase of at least one physical unit.
- **Inactive Case:** If $y_j = 0$, then $x_j \geq 0$, which is naturally satisfied.

Together, these constraints enforce the logic: $y_j = 1 \iff 1 \leq x_j \leq M$.

E. Logical Dependencies (Conditional Constraints)

To prevent the optimizer from selecting food items that cannot be practically prepared or consumed on their own, we enforce strict necessity constraints linking dry staples to required complementary ingredients. We model these dependencies using the following set of linear inequalities:

$$y_{\text{oats}} \leq y_{\text{milk}}, \quad y_{\text{cereal}} \leq y_{\text{milk}}, \quad y_{\text{pasta}} \leq y_{\text{oil}}$$

These constraints follow the form $y_{\text{dependent}} \leq y_{\text{required}}$, representing the logical implication "If Dependent, Then Required."

- **Active Case:** If the dependent item is purchased ($y_{\text{dep}} = 1$), the inequality becomes $1 \leq y_{\text{req}}$, mathematically forcing the required item to be included in the optimal solution.
- **Inactive Case:** If the dependent item is not purchased ($y_{\text{dep}} = 0$), the inequality becomes $0 \leq y_{\text{req}}$, leaving the required item unconstrained (it may still be selected if optimal for other reasons, but is not forced).

2.3 Nature of the Optimization Problem

This problem is a mixed-integer linear program and is therefore non-convex due to the integrality constraints on the decision variables. Although both the objective function (cost) and all constraints are linear, the feasible region consists of a discrete set of integer-valued points and is not convex. As a result of the integer and binary decision variables, the feasible region is disjoint, consisting of isolated feasible solutions rather than a single connected set.

A global optimal solution is nevertheless guaranteed to exist because the feasible set is finite and non-empty. Finiteness follows from the backpack constraint and the Big-M bounds linking the binary and integer variables; non-emptiness is ensured provided that the available food items can collectively meet the minimum energy requirement of 15,750 kcal. Moreover, since all prices are non-negative, the cost objective is bounded below by zero.

2.4 Model Adaptability

The model is intentionally designed to support a wide range of dietary preferences through the selective relaxation or exclusion of constraints. Rather than being tailored to a single dietary profile, the model allows students to adapt the feasible set by modifying category-level requirements or disallowing specific food groups.

For instance, a vegan student may relax the dairy requirement, while a gluten-free or culturally restricted diet can be accommodated by excluding the relevant item categories. In all cases, the structure of the optimization problem remains unchanged; only the constraint set is adjusted.

This constraint-based design demonstrates the model’s flexibility and makes it well suited to the diverse and globally mobile Minerva student population.

3. Data Standardization & Bias

3.1 Data Collection

To evaluate the generalized framework, I curated two parallel grocery datasets representing distinct economic environments from Minerva rotation cities: Scenario A (Buenos Aires) using Coto Digital and Scenario B (San Francisco) using Walmart. Both datasets contain the same fixed list of 24 commonly available grocery items, selected to reflect typical student purchasing behavior. I also chose standard consumer package sizes for each item and kept consistent across both datasets to ensure comparability.

Nutritional information (protein in g and energy in kcal) was obtained through quick reference searches of commonly cited nutrition databases and product packaging. These values were treated as invariant across locations and averaged where necessary, ensuring that observed differences in optimization outcomes arise solely from price variation. This allowed me to investigate the nature of the optimal solutions through a parameter sensitivity analysis by solving the model under distinct conditions: varying the cost vector c (Buenos Aires vs. San Francisco) and modifying the feasible region constraints (Standard vs. Vegan).

3.2 Standardization Protocol

To mitigate selection bias (such as comparing premium products in one city to generic alternatives in another) I adopted a Rational Economic Agent standardization protocol. For each item, the price input corresponds to the lowest available price per unit for a standard package size, identified by selecting the least expensive non-specialty option.

By standardizing item lists, package sizes, and nutritional values across both datasets, the model isolates the effect of local price structures on the optimal grocery basket. This ensures a fair and interpretable comparison of cost-optimal solutions across cities.

4. Analysis and Discussion

4.1 The Universal Student Diet

Across both Buenos Aires and San Francisco, the optimizer produces almost the same dietary structure despite large differences in currency and local prices (see Appendix grocery basket outputs). In both cities, the solution centers on a “Rice and Legumes” base. Rice (1 kg) is selected twice and serves as the main source of calories, as it is the cheapest way to meet the energy requirement. Protein needs are met using dry lentils and dry beans, showing that plant-based legumes provide protein more cheaply than animal or processed options in both locations.

This similarity suggests that the model is driven by basic cost-efficiency rather than city-specific pricing details, resulting in a consistent “universal student diet.”

4.2 Geographic Price Sensitivity

While the core staples remain unchanged, secondary food choices vary according to local prices. In the fruit category, San Francisco selects bananas, reflecting the lower cost of imported tropical fruit, whereas Buenos Aires selects apples, which are locally abundant. A similar pattern appears in dairy: San Francisco selects milk, while Buenos Aires selects yogurt, indicating that yogurt offered a better protein-to-price ratio in the Buenos Aires dataset. These substitutions show that the model responds smoothly to local pricing conditions without altering the overall structure of the diet.

4.3 Structural Cost Allocation

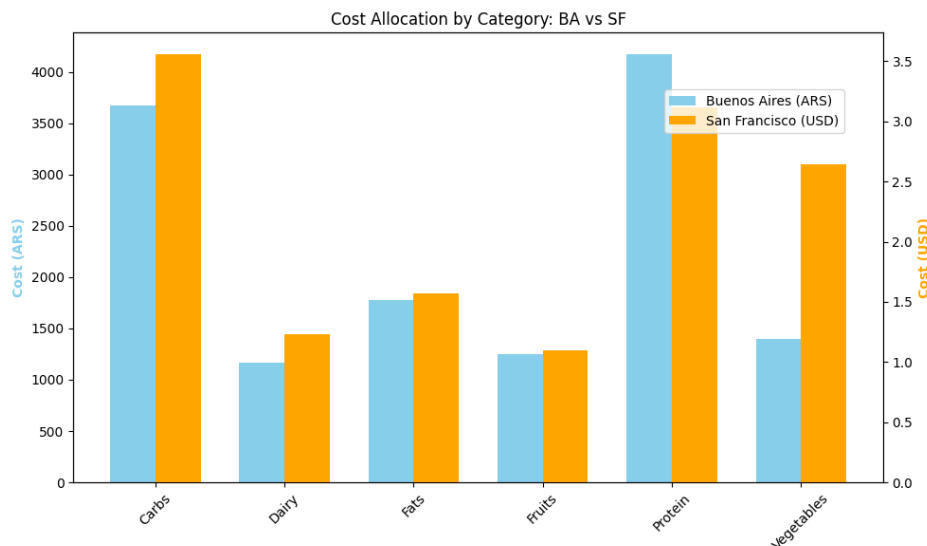


Figure 1: Total weekly spending by food category in the optimal grocery basket for Buenos Aires (ARS) and San Francisco (USD). Despite large differences in currency and absolute prices, both cities exhibit a similar macro-structure: inexpensive carbohydrates supply most calories, while health-driven categories dominate total cost.

Figures 1 and 2 reveal a consistent economic pattern across both cities: carbohydrates and fats form a stable and highly efficient caloric base, while dietary diversity drives total cost. This disparity can be demonstrated directly by comparing each category’s share of total spending to its share of total calories. In the Buenos Aires grocery basket, rice accounts for approximately **27%** of total cost (ARS 3,672 of ARS 13,436) but supplies roughly **46%** of the weekly caloric requirement (7,200 of 15,750 kcal), making it an extremely efficient source of energy. In contrast, vegetables (tomatoes and carrots) consume about **10.4%** of the budget (ARS 1,399) while contributing only **3.2%** of total calories (500 kcal). This implies a stark difference in caloric return on spending where vegetables cost roughly **5.5 times more per calorie** than rice. Similar patterns hold for fruits and dairy, which provide minimal energy at relatively high cost. Together, these results show that total dietary cost is driven less by meeting energy needs than by enforcing nutritional balance. The imposed health constraints require the optimizer to include foods that are inefficient for pure caloric survival, making a healthy, diverse diet structurally more expensive than a calorie-sufficient one in both economies.

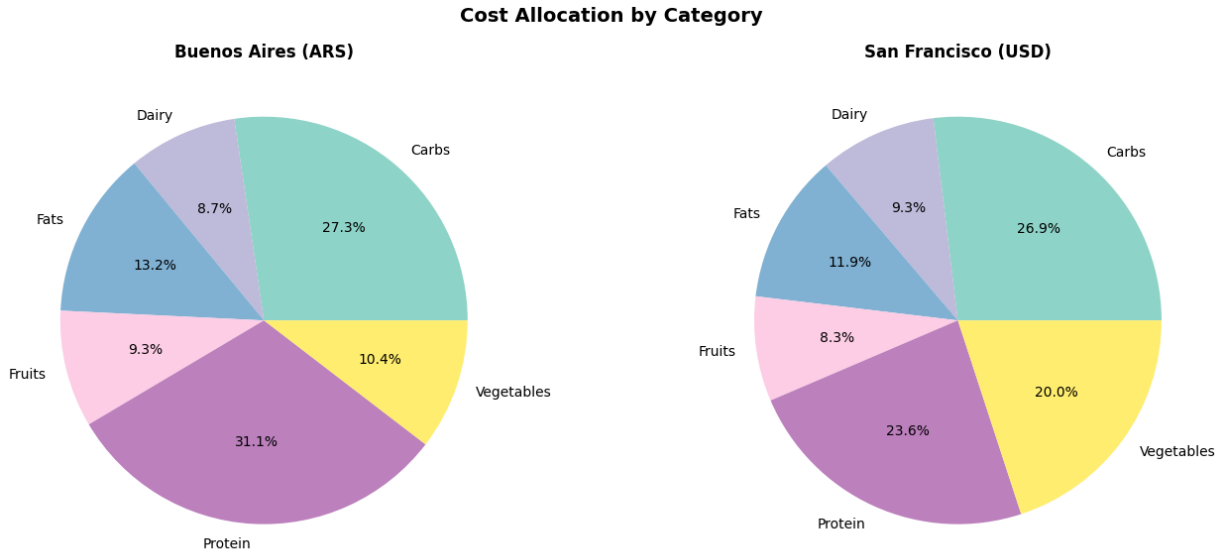


Figure 2: Side-by-side pie charts highlighting city-specific grocery spending. In San Francisco, vegetables account for a disproportionately large share of spending, whereas in Buenos Aires, protein accounts for the largest budget share. In both cases, non-caloric nutritional requirements drive the majority of total cost.

4.4 City-Specific Cost Pressures

Although the overall cost structure is similar, Figures 1 and 2 reveal important local differences. In San Francisco, vegetables are the primary cost driver, consuming nearly **20%** of the total budget, compared to about **10%** in Buenos Aires. This reflects the high relative price of fresh produce in U.S. urban markets. In contrast, Buenos Aires allocates a larger share of spending to protein (approximately **31%**, versus **24%** in San Francisco), driven by the cost of shelf-stable legumes. Despite these differences, both cities devote roughly **60%** of total spending to non-caloric categories, confirming that the main cost of a healthy diet comes from nutrients, not calories.

4.5 Constraint Sensitivity and the Vegan Test Case

Removing the dairy constraint in the vegan sensitivity scenario lowers the total Buenos Aires cost from **ARS 13,436** to **ARS 12,672**. The optimizer simply removes yogurt and relies on existing plant-based staples, without introducing costly substitutes. This shows that animal products are not cost-minimizing under whole-food optimization. Contrary to common assumptions, the model indicates that a vegan diet based on unprocessed foods is mathematically cheaper, and that the perceived expense of veganism largely comes from processed alternatives.

4.6 Limitations and Improvements

While the model effectively minimizes food costs, it treats food only as a source of energy. It does not account for a repetitive menu, taste, cooking effort, or full micronutrient needs, which makes the resulting diet difficult to sustain over time. The solution is therefore financially efficient but not very realistic for everyday life. Future versions of the model could improve realism by discouraging repetitive meals, limiting required preparation time, and including a broader set of micronutrient constraints.

In other words, my model formalizes a familiar reality of the Minerva experience: while achieving the diet my nutritionist mother would approve of is technically possible on a student budget (I failed to prove her wrong!), it appears to require a long-term commitment to rice and legumes, even as the city and prices change every four months.

Appendix A: Dataset Curation Methodology

Dataset Specifications

This appendix documents practical implementation choices I made during dataset construction that are not described in the main text.

Table 1: Dataset Construction Principles

Principle	Implementation
Nutritional Invariance	All nutritional values (energy in kcal and protein in grams) were calculated per full purchase unit and treated as invariant across cities. Values were averaged from multiple sources where consulted.
Price Standardization	Prices extracted from Coto Digital (BA) and Walmart (SF) by selecting the lowest-priced non-specialty option. Private-label/generic brands prioritized. Package size discrepancies adjusted linearly.
Categorical Mapping	Items mapped to six discrete food categories: Carbohydrates, Protein, Vegetables, Fruits, Dairy, Fats to support diversity constraints.
Structural Consistency	Both datasets (<code>prices_ba.csv</code> and <code>prices_sf.csv</code>) contain identical 24 items with identical nutritional values, differing only in price.

Methodological Details

A fixed dataset of 24 grocery items was formed into two parallel CSV datasets (`prices_ba.csv` and `prices_sf.csv`). This approach ensures:

1. **Controlled Comparison:** By holding nutritional values constant, optimization outcomes reflect *only* price differences between cities.
2. **Operational Practicality:** Nutritional values calculated per purchase unit (e.g., per 1kg rice, per 500g lentils) match the integer decision variables in the MILP formulation.
3. **Budget Consciousness:** Price selection follows a "rational economic agent" heuristic—always choosing the lowest available price for a standard consumer package size.
4. **Category-Based Analysis:** Six food categories enable the enforcement of diversity constraints that ensure balanced nutritional intake.

Data Quality Measures

- **Source Transparency:** All prices sourced from publicly accessible online grocery platforms during December 2025, as mentioned later in my References section.
- **Unit Normalization:** Linear adjustment applied to minor package size variations (e.g., 400g $\rightarrow$ 500g).
- **Missing Data:** No items were omitted; complete 24-item list maintained in both datasets.
- **Documentation:** Complete datasets provided in Tables 2 and 3 for verification.

These implementation choices ensure internal consistency, reproducibility, and a clean separation between nutritional parameters and local price structure, allowing for interpretable cross-city comparisons.

Table 2: Buenos Aires Price Dataset

Table 2: Buenos Aires Grocery Prices (Coto Digital)

Item	Price (ARS)	Protein (g)	Calories (kcal)	Category
Rice (1kg)	1,836	70	3,600	Carbs
Pasta (500g)	1,486	60	1,800	Carbs
Oats (500g)	2,862	65	1,900	Carbs
Cereals (500g)	2,488	50	1,850	Carbs
Bread	3,622	45	1,300	Carbs
Potatoes (1kg)	899	20	770	Carbs
Eggs (6)	2,325	36	420	Protein
Chicken breast (500g)	3,824	150	825	Protein
Canned tuna (160g)	1,911	40	190	Protein
Dry lentils (500g)	1,899	125	1,700	Protein
Dry beans (500g)	2,274	105	1,650	Protein
Peanut butter (340g)	4,210	85	2,000	Protein
Milk (1L)	1,547	30	640	Dairy
Yogurt (500g)	1,163	25	300	Dairy
Cheese (200g)	2,125	50	800	Dairy
Spinach (250g)	2,874	7	60	Vegetables
Tomatoes (500g)	500	5	90	Vegetables
Onions (1kg)	899	11	400	Vegetables
Carrots (1kg)	899	9	410	Vegetables
Apples (1kg)	1,249	3	520	Fruits
Bananas (1kg)	1,999	11	890	Fruits
Strawberries (500g)	6,937	4	160	Fruits
Cooking oil (500 ml)	1,780	0	4,000	Fats
Butter (200g)	2,125	2	1,430	Fats

Table 3: San Francisco Price Dataset

Table 3: San Francisco Grocery Prices (Walmart)

Item	Price (USD)	Protein (g)	Calories (kcal)	Category
Rice (1kg)	1.78	70	3,600	Carbs
Pasta (500g)	1.08	60	1,800	Carbs
Oats (500g)	2.82	65	1,900	Carbs
Cereals (500g)	2.92	50	1,850	Carbs
Bread	1.16	45	1,300	Carbs
Potatoes (1kg)	1.09	20	770	Carbs
Eggs (6)	1.24	36	420	Protein
Chicken breast (500g)	3.27	150	825	Protein
Canned tuna (160g)	1.08	40	190	Protein
Dry lentils (500g)	1.56	125	1,700	Protein
Dry beans (500g)	1.56	105	1,650	Protein
Peanut butter (340g)	1.68	85	2,000	Protein
Milk (1L)	1.23	30	640	Dairy
Yogurt (500g)	1.37	25	300	Dairy
Cheese (200g)	1.96	50	800	Dairy
Spinach (250g)	2.63	7	60	Vegetables
Tomatoes (500g)	1.01	5	90	Vegetables
Onions (1kg)	1.80	11	400	Vegetables
Carrots (1kg)	1.63	9	410	Vegetables
Apples (1kg)	1.96	3	520	Fruits
Bananas (1kg)	1.10	11	890	Fruits
Strawberries (500g)	4.93	4	160	Fruits
Cooking oil (500 ml)	1.57	0	4,000	Fats
Butter (200g)	1.74	2	1,430	Fats

Appendix B: Optimal Grocery Baskets

Table 4: Buenos Aires (Standard Diet)

Table 4: Optimal Grocery Basket for Buenos Aires (Standard)

Item	Qty	Cost (ARS)	Category
Rice (1kg)	2	3,672	Carbs
Dry lentils (500g)	1	1,899	Plant protein
Dry beans (500g)	1	2,274	Plant protein
Yogurt (500g)	1	1,163	Dairy
Tomatoes (500g)	1	500	Vegetables
Carrots (1kg)	1	899	Vegetables
Apples (1kg)	1	1,249	Fruits
Cooking oil (500 ml)	1	1,780	Fats
Total		13,436 ARS	8 items

Table 5: San Francisco (Standard Diet)

Table 5: Optimal Grocery Basket for San Francisco (Standard)

Item	Qty	Cost (USD)	Category
Rice (1kg)	2	3.56	Carbs
Dry lentils (500g)	1	1.56	Plant protein
Dry beans (500g)	1	1.56	Plant protein
Milk (1L)	1	1.23	Dairy
Tomatoes (500g)	1	1.01	Vegetables
Carrots (1kg)	1	1.63	Vegetables
Bananas (1kg)	1	1.10	Fruits
Cooking oil (500 ml)	1	1.57	Fats
Total		13.22 USD	8 items

Table 6: Buenos Aires (Vegan Diet)

Table 6: Optimal Grocery Basket for Buenos Aires (Vegan)

Item	Qty	Cost (ARS)	Category
Rice (1kg)	2	3,672	Carbs
Dry lentils (500g)	1	1,899	Plant protein
Dry beans (500g)	1	2,274	Plant protein
Onions (1kg)	1	499	Vegetables
Carrots (1kg)	1	899	Vegetables
Apples (1kg)	1	1,249	Fruits
Cooking oil (500 ml)	1	1,780	Fats
Total		12,672 ARS	7 items

Appendix C: Python Code Implementation

Main Function: Minervan Diet Optimizer (MDO)

```
1 import cvxpy as cp
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 def solve_diet_problem(csv_filepath, is_vegan=False):
8     """
9     Solves the MILP Diet Problem.
10    Args:
11        csv_filepath (str): Path to price data.
12        is_vegan (bool): If True, disables Dairy requirement and Logic constraints.
13    Returns:
14        dict: Solution details (status, total_cost, basket_df).
15    """
16    # 1. Load Data
17    try:
18        df = pd.read_csv(csv_filepath)
19    except FileNotFoundError:
20        return {"status": "File Not Found"}
21
22    # --- DYNAMIC COLUMN MAPPING ---
23    try:
24        price_col = [c for c in df.columns if 'Price' in c][0]
25        prot_col = [c for c in df.columns if 'Protein' in c][0]
26        cal_col = [c for c in df.columns if 'Calorie' in c][0]
27    except IndexError:
28        return {"status": "Error: Columns not found. Check CSV headers."}
29
30    prices = df[price_col].values
31    protein = df[prot_col].values
32    calories = df[cal_col].values
33    names = df['Item'].values
34
35    def is_cat(i, tag):
36        cat_val = str(df.loc[i, 'Category'])
37        return tag.lower() in cat_val.lower()
38
39    n = len(df)
40    M = 50 # Big-M: Max physical units per item
41
42    # 2. Variables
43    x = cp.Variable(n, integer=True) # Quantity
44    y = cp.Variable(n, boolean=True) # Binary Indicator
45
46    # 3. Constraints
47    constraints = []
48    constraints.append(x >= y) # If selected (y=1), must buy at least 1 unit
49    constraints.append(x >= 0) # Non-negativity
50
51    # A. Nutritional Bounds (WHO Average)
52    constraints.append(x @ protein >= 378) # 54g/day * 7
53    constraints.append(x @ calories >= 15750) # 2250kcal/day * 7
54
55    # B. Backpack Constraint
56    constraints.append(cp.sum(x) <= 15)
```

```

57
58 # C. Big-M Linking
59 constraints.append(x <= M * y)
60
61 # D. Category Diversity
62 idx_veg = [i for i in range(n) if is_cat(i, 'Vegetables')]
63 if idx_veg: constraints.append(cp.sum(y[idx_veg]) >= 2)
64
65 idx_fruit = [i for i in range(n) if is_cat(i, 'Fruits')]
66 if idx_fruit: constraints.append(cp.sum(y[idx_fruit]) >= 1)
67
68 idx_prot = [i for i in range(n) if is_cat(i, 'Protein')]
69 if idx_prot: constraints.append(cp.sum(y[idx_prot]) >= 2)
70
71 idx_dairy = [i for i in range(n) if is_cat(i, 'Dairy')]
72 if not is_vegan and idx_dairy:
73     constraints.append(cp.sum(y[idx_dairy]) >= 1)
74 elif is_vegan and idx_dairy:
75     constraints.append(cp.sum(x[idx_dairy]) == 0)
76
77 # E. Logical Dependencies (Strict Necessity)
78 def get_idx(substring):
79     matches = [i for i, name in enumerate(names)
80                if substring.lower() in name.lower()]
81     return matches[0] if matches else None
82
83 if not is_vegan:
84     i_milk = get_idx('Milk')
85     i_oats = get_idx('Oats')
86     if i_oats is not None and i_milk is not None:
87         constraints.append(y[i_oats] <= y[i_milk])
88
89     i_cereal = get_idx('Cereal')
90     if i_cereal is None: i_cereal = get_idx('Cereals')
91     if i_cereal is not None and i_milk is not None:
92         constraints.append(y[i_cereal] <= y[i_milk])
93
94 i_pasta = get_idx('Pasta')
95 i_oil = get_idx('Oil')
96 if i_pasta is not None and i_oil is not None:
97     constraints.append(y[i_pasta] <= y[i_oil])
98
99 # 4. Objective
100 objective = cp.Minimize(x @ prices)
101
102 # 5. Solve
103 prob = cp.Problem(objective, constraints)
104 prob.solve()
105
106 # 6. Package Results
107 result = {
108     "status": prob.status,
109     "cost": prob.value if prob.status == 'optimal' else None,
110     "basket": None
111 }
112
113 if prob.status == 'optimal':
114     df['Qty'] = np.round(x.value)
115     basket = df[df['Qty'] > 0].copy()
116     basket['Total_Cost'] = basket['Qty'] * basket[price_col]

```

```

117     result['basket'] = basket
118
119     return result

```

Listing 1: Python implementation of solve_diet_problem() function

Scenario Execution Code

```

1  # --- Scenario 1: Buenos Aires (Standard) ---
2  res_ba = solve_diet_problem('prices_ba.csv', is_vegan=False)
3
4  print(f"--- BUENOS AIRES RESULTS ---")
5  print(f"Status: {res_ba['status']}")
6  if res_ba['cost']:
7      print(f"Total Cost: ${res_ba['cost']:.2f}")
8      if res_ba['basket'] is not None:
9          cols = ['Item', 'Qty', 'Total_Cost', 'Category']
10         print(res_ba['basket'][cols])
11
12 # --- Scenario 2: San Francisco (Standard) ---
13 res_sf = solve_diet_problem('prices_sf.csv', is_vegan=False)
14
15 print(f"\n--- SAN FRANCISCO RESULTS ---")
16 print(f"Status: {res_sf['status']}")
17 if res_sf['cost']:
18     print(f"Total Cost: ${res_sf['cost']:.2f}")
19     if res_sf['basket'] is not None:
20         cols = ['Item', 'Qty', 'Total_Cost', 'Category']
21         print(res_sf['basket'][cols])
22
23 # --- Scenario 3: Vegan Sensitivity Test ---
24 res_vegan = solve_diet_problem('prices_ba.csv', is_vegan=True)
25
26 print(f"\n--- VEGAN SENSITIVITY TEST (BA) ---")
27 print(f"Status: {res_vegan['status']}")
28 if res_vegan['cost']:
29     print(f"Total Cost: ${res_vegan['cost']:.2f}")
30     if res_vegan['basket'] is not None:
31         print(res_vegan['basket'][['Item', 'Qty', 'Category']])

```

Listing 2: Python code for running the three scenarios

Visualization 1: Bar Chart Comparison

```

1  def plot_results(basket_ba, basket_sf):
2      # Aggregate costs by Category
3      cat_ba = basket_ba.groupby('Category')['Total_Cost'].sum()
4      cat_sf = basket_sf.groupby('Category')['Total_Cost'].sum()
5
6      all_cats = sorted(list(set(cat_ba.index) | set(cat_sf.index)))
7      ba_vals = [cat_ba.get(c, 0) for c in all_cats]
8      sf_vals = [cat_sf.get(c, 0) for c in all_cats]
9
10     x = np.arange(len(all_cats))
11     width = 0.35
12
13     fig, ax1 = plt.subplots(figsize=(10, 6))
14     ax1.bar(x - width/2, ba_vals, width,

```

```

15     label='Buenos Aires (ARS)', color='skyblue')
16 ax1.set_ylabel('Cost (ARS)', color='skyblue', fontweight='bold')
17
18 ax2 = ax1.twinx()
19 ax2.bar(x + width/2, sf_vals, width,
20         label='San Francisco (USD)', color='orange')
21 ax2.set_ylabel('Cost (USD)', color='orange', fontweight='bold')
22
23 ax1.set_xticks(x)
24 ax1.set_xticklabels([c.split(';')[0] for c in all_cats], rotation=45)
25 ax1.set_title('Cost Allocation by Category: BA vs SF')
26
27 fig.legend(loc='upper right', bbox_to_anchor=(0.9, 0.85))
28 plt.tight_layout()
29 plt.show()
30
31 # Generate plot if both solutions are optimal
32 if res_ba['status'] == 'optimal' and res_sf['status'] == 'optimal':
33     plot_results(res_ba['basket'], res_sf['basket'])

```

Listing 3: Python function for comparative bar chart

Visualization 2: Cost Allocation Pie Charts

```

1 def plot_comparative_pies(basket_ba, basket_sf):
2     """Plot side-by-side pie charts for BA vs SF cost allocation."""
3     fig, axes = plt.subplots(1, 2, figsize=(14, 6))
4
5     # Prepare data
6     ba_data = basket_ba.groupby('Category')['Total_Cost'].sum()
7     sf_data = basket_sf.groupby('Category')['Total_Cost'].sum()
8
9     # Get consistent colors
10    categories = sorted(set(ba_data.index) | set(sf_data.index))
11    colors = plt.cm.Set3(np.linspace(0, 1, len(categories)))
12    color_map = dict(zip(categories, colors))
13
14    # BA pie
15    axes[0].pie(ba_data, labels=ba_data.index, autopct='%1.1f%%',
16               colors=[color_map[c] for c in ba_data.index])
17    axes[0].set_title('Buenos Aires (ARS)', fontweight='bold')
18
19    # SF pie
20    axes[1].pie(sf_data, labels=sf_data.index, autopct='%1.1f%%',
21               colors=[color_map[c] for c in sf_data.index])
22    axes[1].set_title('San Francisco (USD)', fontweight='bold')
23
24    plt.suptitle('Cost Allocation by Category', fontsize=14, fontweight='bold')
25    plt.tight_layout()
26    plt.show()
27
28    # Execute if optimal
29    if res_ba['status'] == 'optimal' and res_sf['status'] == 'optimal':
30        plot_comparative_pies(res_ba['basket'], res_sf['basket'])

```

Listing 4: Python function for comparative pie charts

References

1. Theoretical Foundation

Stigler, G. J. (1945). *The Cost of Subsistence*. Journal of Farm Economics, 27(2), 303–314.
(The original paper defining the diet problem as a linear programming optimization).

Dantzig, G. B. (1990). *The Diet Problem*. Interfaces, 20(4), 43–47.
(The classic formulation of the Simplex method applied to Stigler’s data).

2. Nutritional Standards & Guidelines

World Health Organization (WHO). (2007). *Protein and Amino Acid Requirements in Human Nutrition*. WHO Technical Report Series 935.
(Source of the 0.83g/kg daily protein requirement used in Section 2.2).

Scientific Advisory Committee on Nutrition (SACN). (2011). *Dietary Reference Values for Energy*. Public Health England.
(Source of the UK government energy benchmarks: 2,500 kcal for males / 2,000 kcal for females).

World Health Organization. *Healthy Diet Fact Sheet No. 394*.
(Source for general macronutrient diversity guidelines).

3. Nutritional Data (Calorie & Protein Estimation)

U.S. Department of Agriculture (USDA). *FoodData Central Database*.
(The primary database used to standardize nutritional values [Calories/Protein per 100g] for the creation of the `prices_ba.csv` and `prices_sf.csv` datasets).

4. Market Pricing Data

Coto Digital (Argentina). *Daily Grocery Pricing*. Retrieved from cotodigital.com.ar.
(Source for Buenos Aires pricing data).

Walmart (USA). *Grocery Pickup & Delivery Prices*. Retrieved from walmart.com.
(Source for San Francisco pricing data).